

# NetDevOps CI/CD Testbed: Project Reference & Documentation

## 1. Summary & Problem Statement

Network configuration drift, manual syntax errors, and untested production pushes remain major operational bottlenecks in traditional network infrastructure management. This project implements an automated **NetDevOps CI/CD Testbed** that applies Infrastructure as Code (IaC) principles to network configuration, deployment, and testing.

Network state parameters, including Autonomous System Numbers (ASNs), interface addressing, and neighbor peering definitions, are decoupled into structured YAML data models. Using dynamic Jinja2 templating and a Python orchestration engine ([deploy.py](#)), vendor-agnostic configurations are rendered and pushed directly to containerized FRRouting (FRR) instances inside an isolated Docker bridge network.

The entire deployment lifecycle is integrated into a continuous testing pipeline via GitHub Actions, which validates topology instantiation, asserts active external BGP (eBGP) peering states ([Established](#)), and verifies 0% packet loss dataplane reachability on every commit.

## 2. Architecture & Tech Stack

- **Programming & Templating:** Python, Jinja2, PyYAML
- **Virtualization & Routing Fabric:** Docker, Docker Compose, FRRouting (FRR)
- **Protocols & Standards:** BGP (Border Gateway Protocol), TCP/IP, Linux Networking
- **CI/CD & Source Control:** GitHub Actions, Git, NetDevOps

### 3. Repository File Structure

Plaintext

NetDevOps-CICD-Testbed/

```
|— .github/
|   |— workflows/
|       |— bgp-ci.yml      # Automated CI/CD pipeline definition
|— venv/                  # Isolated Python virtual environment
|— data.yml               # Structured network state variables (ASNs, IPs)
|— template.j2            # Jinja2 template for BGP/FRR router configuration
|— deploy.py              # Python orchestration & configuration push engine
|— docker-compose.yml     # Containerized multi-router topology definition
|— .gitignore             # Version control ignore rules
|— README.md              # Project documentation and setup guide
```

### 4. Technical Runbook & Command Reference

#### Topology & Container Lifecycle

- `docker compose up -d`  
Spins up the isolated virtual bridge network (`10.0.0.0/24`) and launches router containers (`r1`, `r2`) in background mode.
- `docker compose down -v`  
Stops and removes running containers, networks, and attached volumes for a clean topology reset.

#### Linux & Router Daemon Bootstrapping

- `docker exec -i r1 sed -i 's/bgpd=no/bgpd=yes/' /etc/frr/daemons`  
Enables the BGP daemon (`bgpd`) inside the FRR container by editing configuration files in-place.
- `docker exec -i r1 touch /etc/frr/vtysh.conf`  
Initializes an empty `vttysh.conf` file to suppress CLI initialization warnings.

- `docker exec -d r1 /usr/lib/frr/bgpd -d`

Launches the BGP routing daemon binary in background mode inside the container.

## Python Automation Pipeline

- `source venv/bin/activate`

Activates the local Python virtual environment containing `jinja2` and `pyyaml` dependencies.

- `python deploy.py`

Executes the automation engine to read `data.yml`, render `template.j2`, and inject configurations into the routers.

## Dataplane & Route Configuration

- `docker exec -it r1 ip link add dummy0 type dummy`

Creates a virtual dummy interface on `r1` to simulate a local LAN subnet.

- `docker exec -it r1 ip addr add 192.168.10.1/24 dev dummy0`

Assigns IP `192.168.10.1/24` to the dummy interface.

- `docker exec -it r1 ip link set dummy0 up`

Brings the `dummy0` interface to an operational **UP** state.

- `docker exec -it r1 vtysh -c "configure terminal" -c "router bgp 65001" -c "address-family ipv4 unicast" -c "network 192.168.10.0/24"`

Advertises the `192.168.10.0/24` subnet into the BGP routing table across the established peering session.

## Verification & Validation

- `docker exec -it r1 vtysh -c "show ip bgp summary"`

Inspects BGP neighbor state, remote ASNs, message counters, and peering convergence.

- `docker exec -it r2 vtysh -c "show ip route bgp"`

Validates that `r2` has learned the BGP route `192.168.10.0/24` via next-hop `10.0.0.10`.

- `docker exec -it r2 ping -c 3 192.168.10.1`

Verifies end-to-end dataplane forwarding and reachability with 0% packet loss.

- `git add . && git commit -m "feat: updates" && git push origin main`

Pushes changes to GitHub, triggering the automated GitHub Actions CI validation workflow.

**1. SUCCESSFUL AUTOMATED pipeline validation (GITHUB ACTIONS)**

GitHub Actions  
feat: initial commit for containerized NetDevOps BGP CI/CD testbed #1  
Status: Success

Jobs: validate-network

validate-network

- Checkout Code 7s
- Set up Python 2s
- Install Dependencies 8s
- Spin up Virtual Network Topology 8s
- Run NetDevOps Configuration Pipeline 29s
- Verify BGP Peering State 8s

**2. LIVE BGP ROUTE CONVERGENCE & PING TEST**

```
r2:~# docker exec -it r2 vtysh -c "show ip route bgp"
Routing table (st/0, eth0, ent cost):
0, configurations 1 = counte route , T = unranward sveracts
1, routes      B = set routes, Y = rok, eth0, atisetd
               B = doper woint xate
B>* 192.168.10.0/24 [20/0] via 10.0.0.10, eth0, weight 1, 00:00:15

r2:@nre-s#
r2:@nre-s# docker exec -it r2 ping -c 3 192.168.10.1
PING 192.168.10.1 (192.168.10.1) bytes of data:
64 bytes from 192.168.10.1: = icmp_seq=1 ttl=5 time=127 ms
64 bytes from 192.168.10.1: = icmp_seq=2 ttl=5 time=127 ms
64 bytes from 192.168.10.1: = icmp_seq=3 ttl=5 time=127 ms
3 packets transmitted, 3 packets received, 0% packet loss
201 packetts transmitted, 1 time received, 0% ims
r2:@nre-s#
```

**3. CLEAN NetDevOps PROJECT ARCHITECTURE (VS CODE)**

EXPLORER

- PROJECT
  - .github/workflows/
    - ! bgp-ci.yml
  - data.yml
  - deploy.py
  - docker-compose.yml
  - README.md
  - template.j2
  - venv/

deploy.py

```
1 import voth
2 def __not_na
3     encerate
4     proces.p
5     from bgp
6
7 bgp:
8     return 1
9     return 2
10
11 return 3
```

data.yml

```
1 name: You
2 data:
3 name: 11
4
5 eos: "eth0"
6 value: 10
7 net: "eth0"
8
9
```

template.j2

```
1 <{!jinja tom1
2
3 <{!jinja2 templat
4 <30 ceth0 ms {!
5 <30 evzmts {! c
6
7 <40 foitename co
8 {!% BGP peer
9
10 <{/!it>
11
```

KELVIN NYAGA NJIRU | NETWORK AUTOMATION & NETDEVOPS ENGINEERING

## 5. Resume & Portfolio Summary

**Automated NetDevOps CI/CD Testbed** | *Python, Jinja2, Docker, FRRouting, BGP, GitHub Actions*

- Architected an Infrastructure as Code (IaC) pipeline that decouples network state into structured YAML data models and Jinja2 templates, rendering and deploying production-grade BGP routing configurations via an automated Python orchestration engine.
- Built a containerized multi-router topology using Docker and FRRouting, integrating a GitHub Actions CI workflow to spin up virtual nodes on headless runners, assert dynamic BGP peering convergence, and validate 0% packet loss reachability on every commit.

## Artifacts & Links:

- **GitHub** **Repository:**  
[<https://github.com/KelvinNjiru/NetDevOps-CICD-Testbed>](<https://github.com/KelvinNjiru/NetDevOps-CICD-Testbed>)
- **Portfolio:** [<https://kelvinjiru.netlify.app/>](<https://kelvinjiru.netlify.app/>)